IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 17 September 2024, accepted 2 October 2024, date of publication 15 October 2024, date of current version 30 October 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3481319

RESEARCH ARTICLE

BDLS as a Blockchain Finality Gadget: Improving Byzantine Fault Tolerance in Hyperledger Fabric

AHMED AL SALIH[1], (Member, IEEE), AND YONGGE WANG[2], (Member, IEEE)
[1]Department of Software and Information Systems (SIS), The University of North Carolina at Charlotte, Charlotte, NC 28223, USA
[2]College of Computing and Informatics, Charlotte, NC 28223, USA

Corresponding author: Ahmed Al Salih (aalsalih@charlotte.edu)

ABSTRACT Decentralized systems are integral to various sectors, including public and private organizations. At the core of these systems is the dissemination protocol. Hyperledger Fabric, a prominent production-ready distribution network system, allows for a pluggable consensus protocol, though it has historically lacked detailed technical guidance for implementing such modules. Initially, Fabric employed Kafka and later switched to Raft, both of which are Crash Fault Tolerant (CFT) protocols and do not support Byzantine fault tolerance (BFT). Hyperledger Fabric recently introduced a new BFT solution called SmartBFT in its 3.0 release. However, SmartBFT, based on the Practical Byzantine Fault Tolerance (PBFT) protocol family, suffers from high message complexity, explicitly referring to the total number of messages exchanged during protocol execution, which scales poorly as the network grows. This complexity significantly affects network efficiency, particularly in large-scale implementations across industries such as healthcare, finance, and education. Communication overhead increases substantially, which encompasses the number of messages and the time and resources required for message transmission and processing. Our research shows that the message complexity of SmartBFT grows quadratically with the number of orderer nodes, leading to significant communication overhead. Conversely, our proposed implementation of our Byzantine Fault Tolerant (BFT) protocol is based on the original Dwork, Lynch, and Stockmeyer (DLS) protocol. Our adaptation is the Blockchain version of DLS (BDLS), which maintains linear message complexity, making it more scalable and efficient for large networks. This stark contrast highlights BDLS's superior scalability and efficiency. BDLS maintains a consistent performance advantage. BDLS throughput analysis of Fabric 3.0 shows a significant performance drop for SmartBFT, which achieves only 40 % and 20 % of Raft's TPS in LAN and WAN environments, respectively. This paper investigates a technical guide for integrating a consensus protocol into Hyperledger Fabric, focusing on integrating the Byzantine Fault Tolerant (BFT) protocol BDLS. Furthermore, the technical efforts and guidelines presented here can serve as a reference for future research aimed at integrating new consensus protocols into Hyperledger Fabric, enhancing the BFT research, and adapting BDLS in different systems.

INDEX TERMS Blockchain, Byzantine fault tolerance (BFT), consensus, distributed ledger technology.

I. INTRODUCTION

The enterprise blockchain market, which includes permissioned platforms like Hyperledger Fabric, is projected to grow significantly from $4.9 billion in 2021 to an estimated $50 billion by 2030, with a compound annual growth rate (CAGR) of 33.6 % reported by *Fortune Business Insights* and *MarketsandMarkets*. This rapid expansion highlights the increasing adoption of blockchain technology in various industries, driven by its ability to provide secure, decentralized, and efficient solutions for business operations. As enterprises scale their blockchain deployments, the demand for robust consensus mechanisms, such as Byzantine Fault Tolerant (BFT) models, will continue to grow to ensure system security, trust, and fault tolerance.

The associate editor coordinating the review of this manuscript and approving it for publication was Kashif Sharif.

The Hyperledger Fabric Ordering service plays a crucial role in ensuring the consistency and reliability of transactions within a Hyperledger Fabric network. This service is responsible for determining the order of all transactions submitted by clients and ensuring that these transactions are committed and broadcast to all network peers. The selection of an appropriate consensus protocol is paramount to achieving these objectives, as it directly impacts the system's performance, security, and fault tolerance.

In contemporary times, organizations across diverse sectors, such as health, education, finance, and beyond, pressingly demand the integration of distributed systems. These systems enhance availability and expedite the delivery of essential services to humanity. Hyperledger Fabric emerges as an ideal solution tailored to meet the unique requirements of disparate industries owing to its versatility and robust functionality. The system's security is an essential factor, as it ensures the trustworthiness and resilience of the distributed ledger against malicious activities and faults.

However, the Hyperledger Fabric Ordering system uses the Raft protocol [1], a crash fault tolerance (CFT) type protocol. It is known for its simplicity and can operate with the presence of half of the Orderer nodes. The Raft consensus algorithm is not aware of malicious transactions or, in other words, not a Byzantine Fault Tolerance (BFT) type protocol, which has become a primary barrier for organizations to participate in a joined ledger because of the lack of security and the absence of a BTF-type consensus algorithm in the Ordering service. Hyperledger introduced Fabric 3.0 in 2023, an enhanced protocol based on the PBFT protocol, which has challenges regarding scaling and performance.

This paper presents a comprehensive analysis of the BDLS consensus protocol, focusing on its application in blockchain systems. The key contributions include introducing an optimized message complexity model that enhances the scalability of BDLS in comparison to traditional Byzantine Fault Tolerant (BFT) protocols. Additionally, we provide detailed performance results, demonstrating that BDLS achieves a significantly higher throughput in transactions per second (TPS) under realistic network conditions. Specifically, our experiments indicate that BDLS reaches 90 % to 95 % of Raft's TPS, outperforming existing BFT solutions in similar environments. These results underline the potential of BDLS for improving the efficiency of large-scale, production-grade blockchain deployments.

A. MOTIVATION

Integrating the Byzantine Fault Tolerance (BFT) protocol BDLS into Hyperledger Fabric Orderer presents significant opportunities for advancing distributed systems across various industries. Given the current limitations of the Raft protocol and the modest performance of Fabric 3.0's SmartBft, there is a clear need for a more robust and high-performance BFT solution. BDLS meets the crucial consensus properties of liveness and safety and demonstrates superior throughput (TPS) compared to existing BFT solutions. This makes BDLS an attractive alternative for industries that require high-speed, secure, and reliable distributed systems.

All code is available in the GitHub repository: https://github.com/hyperledger-labs/bdls.

II. HYPERLEDGER FABRIC ORDERER

The Orderer node in Hyperledger Fabric [2] is a critical component responsible for ordering and validating transactions before committing to the blockchain. It provides a distributed ordering service that ensures that all nodes in the network have a consistent view of the order of transactions. The Orderer node maintains the ledger's integrity and guarantees that transactions are not tampered with or duplicated.

The Hyperledger Fabric Orderer node uses a consensus mechanism to order transactions. There are several consensus mechanisms available, including Solo [2], Kafka [3], and Raft [1] as of Fabric's latest release 2.5 [4]. Solo is a simple consensus mechanism suitable for testing and development environments, whereas Kafka and Raft are more robust and suitable for production environments.

The Orderer in Hyperledger Fabric is designed to be modular and pluggable [5], which means that organizations can choose the consensus mechanism that best suits their needs. Additionally, the Orderer can be run as a standalone service or part of a more extensive network.

In Hyperledger Fabric, individual nodes can submit transactions to the Orderer. The Orderer receives these transactions, assembles them into blocks, and ensures consensus among all Orderer nodes. Once consensus on a block is achieved, the block is distributed to peers.

A. CONSENSUS IN FABRIC ORDERER

The consensus service in Hyperledger Fabric is designed to be a pluggable module, allowing for the flexibility to switch between different consensus options based on specific application scenarios. In the latest version of Hyperledger Fabric, the consensus module incorporates three implemented consensus algorithms: Solo, Kafka, and Raft. Whereas the official recommendation is to utilize the Raft consensus algorithm, it is essential to briefly introduce the Solo and Kafka consensus algorithms to gain a comprehensive understanding of the consensus module in Fabric.

1) SOLO

The Solo consensus algorithm assumes a network environment with a single ordering node. In this scenario, the messages sent from peer nodes are sorted. The single ordering node receives the messages and creates the block. Although Solo ensures sequential consistency, it lacks high availability and scalability. Consequently, it is unsuitable for production environments and is primarily used in development and testing environments.

2) KAFKA

The Kafka is a distributed streaming information processing platform designed to offer unified, high-throughput, and low-latency performance for real-time data. The previous version of Hyperledger Fabric implemented the core consensus algorithm through a Kafka cluster. In essence, all transaction information is sorted through Kafka, with each channel being sorted independently if multiple channels exist within the system.

3) RAFT

Hyperledger Fabric integrated the Raft consensus algorithm in version 1.4.1. Based on the etcd based crash fault tolerance (CFT) ordering service, Raft follows a ''leader and followers'' model. A leader is dynamically elected within a channel from the Orderer nodes, known as the *consenter set*. The leader replicates messages to follower nodes. Raft ensures the system can provide services to the external world even in the presence of failure nodes, $(N-1)/2$ Where N is the number of Orderer nodes.

4) SMARTBFT

Hyperledger Fabric 3.0 has recently announced the adoption of the Byzantine Fault Tolerance (BFT) protocol, specifically Smart BFT [6], [7]. However, this version has not yet been officially released. This protocol's implementation and experimental analysis will be discussed in subsequent sections. The current stable release from the Hyperledger Fabric organization is version 2.5.

B. PLUGGABLE CONSENSUS ALGORITHM

Hyperledger Fabric Orderer node is a pluggable consensus protocol [8]. There is a standard set of operations in order to plug any algorithm protocol. This includes defining the new protocol in the Orderer `main` function and implementing core interfaces required to operate the Orderer node and handle the different types of messages. This paper provides an in-depth guide through the technical steps necessary to integrate any consensus protocol.

The `main` function is the starting point of the Fabric Orderer node. The main function reads and loads the configuration artifacts from the YAML file. The *initializeMultichannelRegistrar* function Initialize and returns the *registerar*. The implementation of the chain and consenter interfaces will be executed during the operation of Initializing the *registerar*. For demonstration, see Figure 1, main functions execute multiple functions. In the *initializeMultichannelRegistrar* function, to return *registerar*, a load triggers the reference by Register then *ChainSupport*. ChainSupport must execute two functions. The first `HandleChain` function in the Consenter interface implementation, this function returns a `NewChain` function located in the chain file. `NewChain` is responsible for initializing the Orderer node requirements. Chain is the chain interface implementation for a specific consensus protocol. The results of executing the `HandleChain` function, the chain initialized successfully. Second, run the Start function in the chain file for the chain interface implementation. This will make the single Orderer node running and ready to receive a message request and participate in the consensus team's decision-making.

C. ORDERER MESSAGES LIFECYCLE

The Hyperledger Fabric system's ultimate function is to receive a message from the node. The Orderer node is then responsible for sorting the transaction, creating the block, and broadcasting the block over the network nodes.

1) BROADCAST

The Broadcast function returns *Handle* function that reads requests from a broadcast stream, processes them accordingly, and sends the corresponding responses back to the stream. This allows bidirectional communication between the server and the clients through the broadcast stream.

The Broadcast, processes normal messages and configuration messages separately, ensuring that each type of message is handled appropriately and in accordance with its specific requirements. This segregation allows for efficient and accurate system processing of different message types.

2) ORDER AND CONFIGURE

The (Order/Configure) functions are the Orderer functions that handle the messages submitted by the client application. The client submits a request message to the Orderer. The Orderer that runs the Raft protocol forwards the received envelope to the leader Orderer node in the raft cluster. The *Order* function handles normal messages, whereas the *Configure* function handles configuration messages. The functions are expected to receive an envelope object of type *common.Envelop* and a number represent the *configSeq*. In Raft implementation, both functions encapsulate the envelope message in *SubmitRequest* struct, then send the new object to the *Submit* function. It is up to the consensus implementer to choose to rap the received message. As demonstrated in Listings 1 & 2.

```
1 // Order submits normal type transactions for
      ordering.
2 func (c *Chain) Order(env *common.Envelope,
      configSeq uint64) error {
3   return c.Submit(&orderer
4     .SubmitRequest{LastValidationSeq: configSeq,
      Payload: env, Channel: c.channelID}, 0)
5 }
```

LISTING 1. Order() and Configure() functions in etcdraft/chain.go.

3) SUBMIT

The *Submit* function encapsulates the request message into a *Submit* struct, as illustrated in Listing (**4**), and subsequently passes it to the backend for processing. This request is sent through the channel *c.submitC*.

As observed, both the *Order* and *Config* functions propose the block to the *Submit* function.

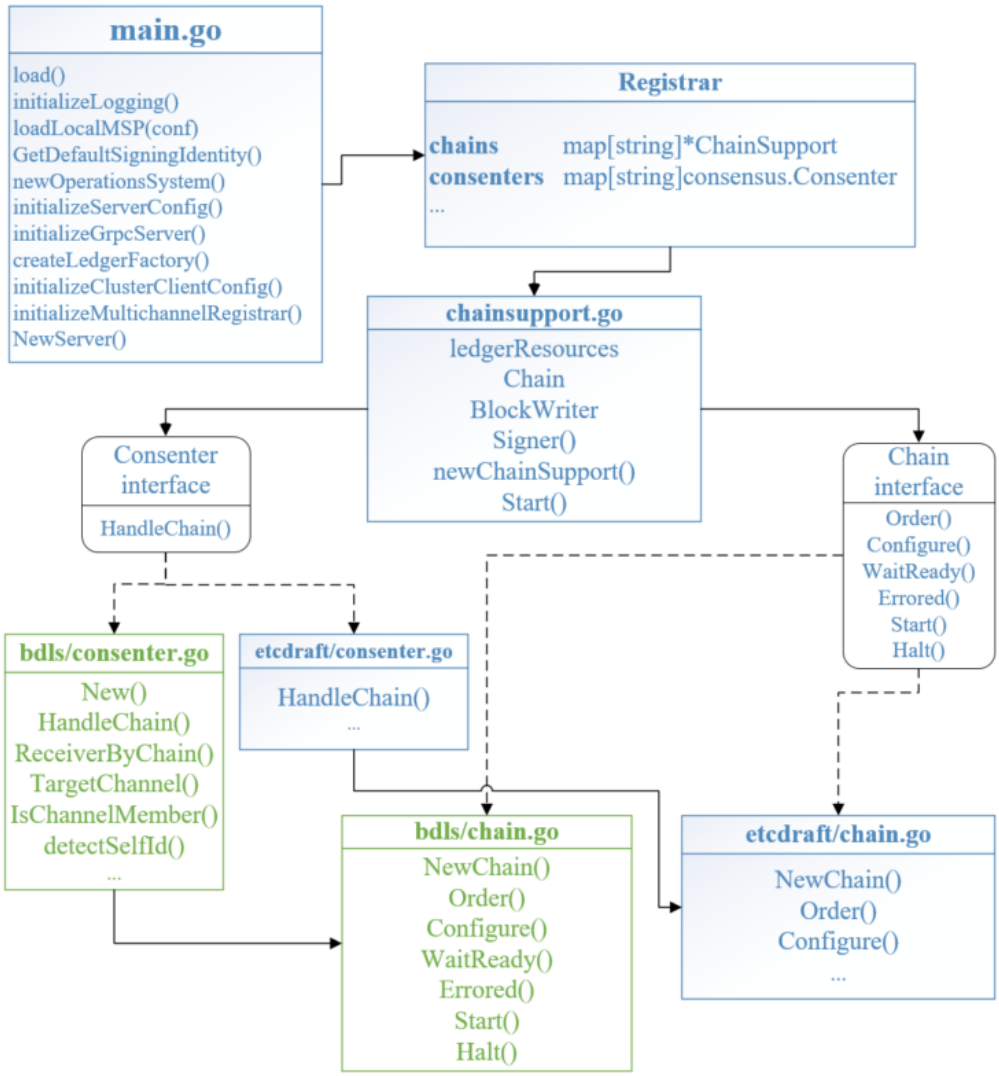


FIGURE 1. Fabric Ordering Service source code, highlighting the primary Golang files and interfaces (Raft/BDLS). The dotted lines indicate the implementations of the interfaces. Green boxes illustrate the core BDLS-Fabric implementation.

```
1
2 // Configure submits config type transactions for
      ordering.
3 func (c *Chain) Configure(env *common.Envelope,
      configSeq uint64) error {
4   return c.Submit(&orderer
5     .SubmitRequest{LastValidationSeq: configSeq,
      Payload: env, Channel: c.channelID}, 0)
6 }
```

LISTING 2. Order() and Configure() functions in etcdraft/chain.go.

```
1 type submit struct {
2   req    *orderer.SubmitRequest
3   leader chan uint64
4 }
```

LISTING 3. Submit struct in orderer/consensus/etcdraft/chain.go.

The *submit* struct, as indicated in Listing 4, encapsulates the *SubmitReques*. The new *submit* object, which holds the payload, is then passed to the global channel *submitC*. The application listens for a response from the *leadC* channel. The envelope will transfer to the raft leader node if the current node is not the leader by the *forwardToLeader* function, as indicated in Listing 4-line(5).

```
1 func (c *Chain) Submit(req
      *orderer.SubmitRequest, sender uint64) error {
2   case c.submitC <- &submit{req, leadC}:
3     lead := <-leadC
4     if lead != c.raftID {
5       if err := c.forwardToLeader(lead, req); err
      != nil {...}
```

LISTING 4. Submit() function in orderer/consensus/etcdraft/chain.go.

III. RELATED WORKS

A. BFT-FABRIC REVIEW

1) PBFT

Hyperledger Fabric incorporates several consensus protocols. Starting in 2015, IBM initiated a project called Open Blockchain (OBC) [9], [10], which utilized the Practical

Byzantine Fault Tolerance (PBFT) protocol [11]. In 2016, the code evolved to be open source. The Fabric source code moved from the OBC-IBM repository to a Hyperledger Foundation repository, named Fabric v0.5-developer-preview [5], [10], still using the PBFT protocol [11]. However, the BFT module was never released for production due to its slowness and performance issues [12], [13]. The primary challenge with PBFT and similar protocols lies in their message complexity and the substantial network traffic generated during message validation operations, severely limiting scalability. Refer to Figure 4 for a detailed view of the PBFT message complexity.

2) BFT-SMART

BFT-SMART's message pattern in typical scenarios is similar to the PBFT protocol, lacking support for transaction pipelining. In BFT-SMART, only a single transaction can be proposed by a leader at any point in time. In the first proposal 2018, Sousa et al. attempted to integrate the protocol to enable a BFT ordering service, replacing the Kafka service with a cluster of BFT-SMART-based servers [14]. However, this approach was not adopted by the community due to both fundamental and technical issues, such as high message complexity and poor performance, since BFT-SMART is PBFT enhanced protocol.

3) SMART-BFT

In September 2023, Hyperledger Fabric announced the new Orderer version 3.0, which uses a BFT algorithm but is not an official release. Furthermore, as of 2024, the recent stable release of Fabric is version 2.5, which maintains Raft consensus algorithms and no BFT model. However, Hyperledger Fabric announced the Byzantine Fault Tolerant (BFT) ordering service. This protocol still suffers from the same limitations as PBFT algorithm protocols, as indicated by using the same message pattern, resulting in high message complexity. Recent results show that this implementation achieves only 20 % of Raft's throughput in WAN environments and 40 % in LAN environments [7], demonstrating significant performance drawbacks.

B. FABRIC V3.0 LIMITATION

1) MESSAGE COMPLEXITY

Hyperledger Fabric recently announced a new BFT solution in its first release to incorporate Byzantine Fault Tolerance (BFT) [15]. Fabric's initial design intended to use Practical Byzantine Fault Tolerance (PBFT). However, Fabric 3.0 embedded the SmartBFT consensus library, which belongs to the PBFT family and suffers from similar message complexity issues as the network grows. In Figure 4, we listed the common BFT protocols proposed for Fabric and their message complexity, including HotStuff [16] adopted in Facebook's Libra [17] blockchain, required only seven steps as we reference in our previous work [18], whereas BDLS required only four steps.

This limitation undermines the goal of maintaining a securely distributed network among institutions. For instance, healthcare systems spanning multiple hospitals at the country, state, or city level cannot effectively incorporate Fabric 3.0. Similar constraints apply to financial, retail, education, and other sectors where multiple organizations must join to share a ledger. In BFT-SMART, the message pattern typically mirrors that of the PBFT protocol, encompassing the *PRE-PREPARE*, *PREPARE*, and *COMMIT* phases/messages. Analyzing the messages required in the network for each step: - PRE-PREPARE: One node sends the transaction to all nodes, represented as n (the number of Orderer nodes). - PREPARE: All nodes send the transaction to all nodes, resulting in n^2 messages. - COMMIT: Similarly, all nodes broadcast to all nodes, resulting in n^2 messages. Similar to MirBFT and other PBFT branches. Thus, the total messages required to achieve consensus for one transaction is $2n^2 + n$.

For a network containing 100 Orderer nodes, submitting a single transaction (i.e., creating a single block) requires:

$$\text{Total Messages} = 2(100)^2 + 100 = 20,100$$

This indicates that hundreds of orderer nodes can significantly burden the network traffic, causing delays. For the minimum network requirement of four orderer nodes in Fabric 3.0 BFT, the messages required to write a single transaction are:

$$\text{Messages} = 2(4)^2 + 4 = 36$$

Our research presents BDLS, which preserves message complexity irrespective of the growth in the number of ordering system nodes. Demonstrating with 100 Orderer nodes, BDLS maintains efficiency. In this example, P_i is the leader node, and *OSNs* are all other nodes in the ordering system. Both P_i and *OSNs* belong to the ordering system.

$$P_i \leftarrow \text{Tx from 100 OSNs}$$
$$P_i \xrightarrow{\text{broadcast}} \text{100 OSNs}$$
$$\text{100 OSNs} \xrightarrow{\text{send}} P_i$$
$$P_i \xrightarrow{\text{broadcast}} \text{100 OSNs}$$

In each step, BDLS utilizes 100 messages sent to or broadcast by the leader to the other nodes. The total transactions for the BDLS four steps is $4n$, resulting in:

$$\text{Total BDLS Messages for 100n} = 4 \times 100 = 400$$

Comparing BDLS's 400 messages to Fabric 3.0 BFT's 20,100 messages illustrates a significant reduction in network load.

For a simple network with at least four orderer nodes, the steps required to write a single transaction using BDLS are:

$$\text{Total BDLS Messages for 4n} = 4 \times 4 = 16$$

In contrast, Fabric 3.0 requires 36 messages. Refer to Figure 2 for a detailed explanation of the message complexity

in PBFT and its derivative protocols. This complexity arises especially during phase 2 (prepare) and phase 3 (commit), where every node communicates with all other nodes.

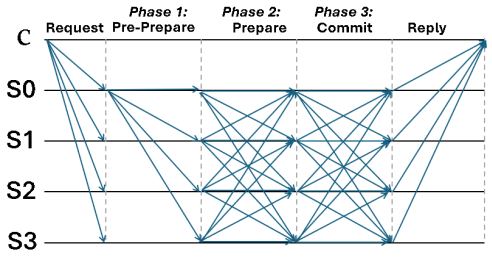


FIGURE 2. PBFT consensus algorithm - PBFT-Hyperledger Fabric full lifecycle data-flow.

2) TPS PERFORMANCE LIMITATION

The throughput results of Fabric 3.0, based on the recently published paper [6] and the Fabric 3.0 announcement homepage, demonstrate significant performance limitations. In a LAN environment, SmartBFT achieves only 20% of Raft protocol performance (2,500 vs. 13,000 transactions per second) [6]. In a WAN environment, SmartBFT achieves 40 % of Raf performance (1,200 vs. 3,000 transactions per second) [6]. This is because SmartBFT [6], which is based on the Practical Byzantine Fault Tolerance (PBFT) algorithm, requires multiple rounds of communication between all pairs of nodes to ensure agreement on the order of transactions and to tolerate Byzantine faults. This communication pattern results in a quadratic increase in the number of messages as the number of nodes increases. However, not all BFT protocols suffer from these issues. Many researchers are constrained by the exact protocol implementation that suffers from message complexity, as listed in popular protocols proposed for Fabric in Figure 4. On the other hand, BDLS utilizes a digital signature scheme for cryptographic operations and message verification, avoiding these pitfalls. The BDLS TPS results demonstrated in our experimental section significantly improve performance. BDLS achieves 95 % of Raft's throughput, illustrating that with proper implementation, BFT protocols can be efficient and secure. This highlights the potential of BDLS as a superior alternative for achieving high performance and robust fault tolerance in Hyperledger Fabric.

3) DENIAL OF SERVICE (DOS) ATTACK

Proven by Wang in [18] that a PBFT type protocol cannot achieve liveness in Type II networks, which is precisely Tendermint BFT [19]. Showing several attacks that can suspend and lockdown will cause the network to reach a deadlock state and cannot achieve consensus. We picked the Tendermint BFT protocol, which is based on PBFT and is a popular modern one. Presenting the message complexity and reduces the authenticator complexity to O(n) utilizing threshold cryptography in Figure 4.

IV. CONTRIBUTION

A. BDLS INTRODUCTION

The Byzantine fault-tolerant (BFT) consensus protocol is a fundamental component of distributed systems. Reference [20] such as blockchain systems, enabling distributed networks to agree on ordering and validating transactions without relying on centralized authorities. Despite its many advantages, the BFT protocol faces significant challenges in terms of scalability, efficiency, and security, particularly in large-scale networks with a high degree of Byzantine faults.

In partly synchronous networks, we suggest a Byzantine Agreement Protocol that achieves safety and liveness features. We could apply this approach to other circumstances, such as state machine replication (SMR). We introduce the protocol as a blockchain finality gadget. Let's assume that our BFT finality gadget has a distinct block proposal mechanism that generates child blocks for finalized blocks.

We're assuming there are $n = 3t + 1$ participants labeled $P_0, \ldots, P_{n-1}$ in the Byzantine Fault Tolerance (BFT) protocol, with at most t of them being malicious. Additionally, each participant has a public-private key pair, with their public key known to all participants. We'll use the notation $\cdot i$ to indicate that a message is digitally signed by participant P_i.

The data flow of the proposed transaction into the BDLS node replicas requires four phases. Figure 3 illustrates the complete consensus process, which includes four steps: *propose*, *lock*, *commit*, and *decide*.

The BDLS protocol is established by systematically proving a series of lemmas, as detailed in previous work [18]. The following points explain the consensus process protocol, which is detailed in Section V, as depicted in Figure 3.

1) Request: The client is responsible for submitting the transaction to all BDLS nodes to initiate the decision-making process on the proposed block. The BDLS protocol steps commence once the client submits the block to all BFT replica members (BDLS nodes), or at least the minimum number of live replicas required by the BFT protocol, which is $3f + 1$ total nodes.
2) **Phase 1** *propose* is the first phase, all BDLS participants P_j (Includes leader P_i) sends a signed message:

$$\langle h, r\rangle_j, \langle h, r, B'_j)_j\rangle$$

to the leader P_i, where $B'_j \in \text{BLOCK}_j$ represents the highest eligible candidate block for P_j. The message $(h, r)_j$ is designated as a round-change message. Following the transmission of the round-change message, P_j ceases to accept any further messages, except for a "decide" message on a round $r' < r$.
In the context of this research, let h denote the height, r denote the round number, and B' denote the candidate block.
3) **Phase 2** *lock*, the leader P_i receives total messages from at least 2t + 1 participants, each containing the

same candidate block in signed messages. The leader then updates the block flag by adding the *lock* keyword and broadcasts the message to all participants. All participants p_j (including the leader p_i) must receive this message from the leader p_i.

$$\langle \text{lock}, h, r, B', \text{proof} \rangle_i$$

A constructed digital signature on the message: $\langle h, r, B' \rangle$ represents the "proof".

4) **Phase 3** *commit* All participants P_j (including the leader P_i) receives message: $\langle \text{lock}, h, r, B', \text{proof} \rangle_i$ Subsequently, all participants P_j (including the leader P_i) update the block flag, adding the *commit* keyword. Then, each participant sends the updated block flag to the leader P_i with the message:

$$\langle \text{commit}, h, r, B' \rangle_j$$

5) **Phase 4** *decide* The leader P_i receives *commit* messages from at least $2t + 1$ total participants. Then:
   a) P_i decides on the value B' by update the flag from *commit* to *decide*.
   b) The leader P_i broadcasts the decided message to all participants, including himself.

$$\langle \text{decide}, h, r, B', \text{proof} \rangle_i$$

At this stage, the system is reaching consensus, indicating that writing the message or block to the local system is appropriate.

6) The replay step notifies the client or system of the committed block that all BDLS participants have decided on. The current BDLS protocol does not notify the client. Instead, it is the client's responsibility to call the *CurrentState()* function, which returns the last *state* (block), *block height* and *round* for checking new blocks. This functionality is successfully implemented in Hyperledger Fabric, as detailed in the *Implement Consenter Interface* section IV-B4, specifically within the **GetLatestState** function.

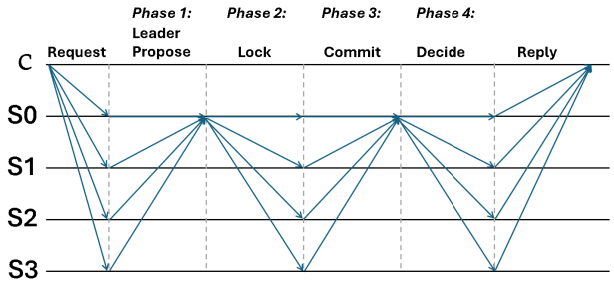


FIGURE 3. BDLS consensus algorithm - BDLS-Hyperledger Fabric full lifecycle data-flow.

The BDLS protocol [18] is one of the most promising consensus BFT protocols. The BDLS protocol is an enhancement to the traditional BFT consensus protocol based on the DLS protocol proposed by Dwork, Lynch, and Stockmeyer for Type II networks [21] that reduces the number of communication rounds required for consensus, thereby increasing efficiency and scalability while maintaining a high degree of security [22].

The BDLS protocol achieves its performance gains by leveraging distributed ledger signatures to reduce the number of signature verifications required during the consensus process. This technique allows the protocol to reach consensus in fewer rounds, reducing the overall time and resource requirements for achieving agreement [22].

B. BDLS-HYPERLEDGER FABRIC INTEGRATION

To integrate the BDLS protocol, it was necessary to develop comprehensive interfaces encompassing both core and utility functions. This section provides a technical overview of integrating BDLS as a consensus protocol for the Fabric, along with the required code change and providing the code repository for references and practices. The change required to integrate the consensus protocol, especially BDLS, into Hyperledger Fabric applied not only to the Hyperledger Fabric core system but changes required in the test network project and the SDK project.

Hyperledger Fabric core project [23]. The change required to integrate a consensus protocol is a heavy technical effort, as it involves importing the consensus protocol package and adding the particular consensus implementation.

Regarding the changes in the Hyperledger fabric-samples project [24] and the Hyperledger fabric-sdk project [25], the new Orderer 3.0 release provided the necessary modifications for the Fabric SDK and fabric-samples to support the BFT-type consensus protocol. This protocol requires that transactions be submitted to the quorum nodes. This change is necessary because the BDLS and all BFT-type consensus protocols require the client to submit transactions to all live nodes, specifically all Orderer nodes.

In the following sections, we will outline the necessary changes to the Fabric Core project to build an Ordering system that supports BDLS. This material can also serve as a guide for integrating other consensus protocols for future experiments.

1) ORDERER MAIN FUNCTION CHANGES

The main function is the starting point of the Orderer node. The selected consensus protocol will be utilized to process incoming messages by incorporating the ability to select the BDLS as a consensus protocol. The Orderer node within the Hyperledger Fabric network can establish and maintain its consensus mechanism during the initialization of the network. This enhancement empowers users to choose BDLS as the preferred consensus protocol for the Hyperledger Fabric orderer. Implementing this change involves including the key-value pair 'bdls' within the variable section. See code in Listing 5, indicate a *map[string]struct* in the *clusterTypes*

Source code: orderer\common\server\main.go *Line70* [26].

Steps	PBFT	Tendermint BFT	HotStuff BFT	BDLS	SmartBFT	MirBFT
1						
2						
3						
4						
5						
6						
7						
message complexity	$2n^2+n$	$2n^2+n$	7n	4n	$2n^2+n$	$2n^2+n$
authenticator complexity	O(n2)	O(n)	O(n)	O(n)	O(n2)	O(n2)

: Leader broadcasts

: All participants send messages to the leader

: All participants broadcast

FIGURE 4. BFT protocols message complexity.

```
var (
 clusterTypes = map[string]struct{}{
      "etcdraft": {},
      "bdls":     {},})
```

LISTING 5. Fabric Orderer main.go-var section.

2) NEW CONSENSUS INFRASTRUCTURE

To incorporate the BDLS protocol's logic and initialize the necessary consensus configuration for starting the consensus and processing of received messages, a new folder named *"bdls"* is created within the Fabric core project.
This folder serves as the repository for the BDLS package and is located at the HLF project directory path:

orderer/consensus/bdls

Writing the code implementation of the consensus protocol requires creating a folder associated with the new protocol for best practices, code isolation, and packaging. This designated folder will be the package name decoration for the Go files within the *bdls* directory. The *bdls* folder acts as the root directory for the implementation files of the BDLS consensus. All files created within this folder are considered members of the bdls package.

3) IMPLEMENT CONSENTER INTERFACE

Create a new **consenter.go** file in the bdls directory as follows:

orderer/consensus/bdls/consenter.go

The primary objective is to create the *HandleChain* function, which is the only function that needs to be overridden to implement the *Consenter interface*. This function returns another function, *NewChain*, responsible for creating the New Chain object from the *chain.go* file, as elaborated in the following section.

Additionally, create the **New** function responsible for creating the BDLS Consenter. The *New()* function is the initial consensus-related function invoked during the orderer execution. Within the Orderer *main.go* file, the *initializeMultichannelRegistrar* function passes the required parameters to the *New* function to create the BDLS Consenter. The BDLS consenter object must then be passed to the *registrar.Initialize(consenters)*.

```
consenters["bdls"] =
    bdls.New(dpmr.Registry(), signer,
    clusterDialer, conf, srvConf, srv,
    registrar, metricsProvider,
    clusterMetrics, bccsp)
registrar.Initialize(consenters)
return registrar
```

4) IMPLEMENT CHAIN INTERFACE

Create a new file named *chain.go* within the newly created *bdls* folder as follows:

orderer/consensus/bdls/chain.go

Create an implementation of all the functions in the Chain interface body to implement the Chain interface.

Order function is essential for receiving the normal message from the Orderer node to a consensus protocol. The 1st parameter this function receives is the *env* type of *Envelope*.

The envelope is a wrapper for a payload, incorporating a signature to ensure message authentication. Contracted in this *proto* file: *vendor/github.com/hyperledger/fabric−protos−go/common/common.pb.go*

Configure function In Hyperledger Fabric, the Configure function is responsible for managing and updating channel configurations. It enables modifications to critical settings such as access control policies, membership, and network parameters. Through the Configure function, administrators can perform configuration transactions that are endorsed, signed, and committed by network participants, ensuring decentralized governance. This function supports dynamic network evolution by facilitating changes like adding new organizations or updating channel policies while maintaining the integrity and security of the blockchain.

NewChain function is to create a chain object. This function is utilized as the return value from the *HandleChain* function within the research context. *NewChain* instructs the orderer to initiate the process of serving the chain and maintaining its up-to-date state. The *configureComm* function can successfully join the list of nodes as parameters within the *Configurator interface*. If the verification fails, an error is returned. Following this, a BDLS config object is created, and the necessary fields are set accordingly to initiate the protocol. These steps ensure the communication layer's proper configuration and the BDLS protocol's initialization.

```
//BDLS consensus config
  config := &bdls.Config{
    Epoch:         time.Now(),
    CurrentHeight: c.lastBlock.Header.Number - 1,
    //Fabric block height ....}
config.Participants = append(config.Participants,
    c.participants...)
```

LISTING 6. BDLS protocol config object creation.

Start function by invoking the Start function, execute both (*run* and *startConsensus*) functions in *goroutine*, a light type of thread managed by the Go runtime.

```
go c.startConsensus(c.config)
go c.run()
```

5) C.STARTCONSENSUS(...)

function it starts the BDLS consensus object by passing the config object created by the *NewChain* function to the *NewConsensus* function as demonstrated below.

```
// Create BDLS consensus Object
consensus, err := bdls.NewConsensus(config)
// Set the BDLS consensus Latency time
consensus.SetLatency(200 * time.Millisecond)
```

LISTING 7. New BDLS consensus object creation.

We assembled an *updateTick* timer run every 20 ms and an infinite loop that listens to the updateTick. The primary goal is to pass the current time of the node to a core BDLS function called *Update*, as the BDLS consensus mandates. Within the *updateTick* channel, it includes the check for the new block when it achieves consensus by calling a BDLS core function *CurrentState()*. This returns three parameter values: height number, current round, and the latest state.

```
  updateTick := time.NewTicker(updatePeriod)
  for {
    <-updateTick.C
    c.consensus.Update(time.Now())
    // Check for confirmed new block
    height ,round, state :=
    c.consensus.CurrentState()
        //Comparing Fabric Height with BDLS Height
    if height > c.lastBlock.Header.Number {
      go func() {
//'state' is the block achived consensus
        c.applyC <- apply{state}
    }()}}
```

LISTING 8. Utilazion BDLS consensus functions (Update & CurrentState).

Comparing whether the height value returned by BDLS is greater than the current height indicates that a new block has achieved consensus in the BDLS protocol. Additionally, it returns the state representing the proposed data (marshaled block) in binary format, which has achieved consensus. The new state returned by BDLS is encapsulated in the apply struct and sent to the *applyC* channel. Ultimately, the *c.applyC* channel sends the state, which is the marshaled value of the proposed block, to the *apply* function. In the *apply* function, we unmarshal the state to retrieve the original block and pass it to the *writeBlock* function.

```
func (c *Chain) apply( height uint64, round
     uint64, state bdls.State) {
  newBlock :=
    protoutil.UnmarshalBlockOrPanic(state)
  c.writeBlock(newBlock, 0)
  c.Metrics.CommittedBlockNumber
    .Set(float64(newBlock.Header.Number))
}
```

LISTING 9. Apply functions (Unmarshal & writeBlock).

6) C.RUN()

function operates within a separate thread (Goroutine). It encapsulates the complete logic required for transaction handling, including submission for the ordering, proposing transactions to the consensus protocol, and writing blocks to the ledger. This function also manages various properties, such as tickers and timers, and executes an anonymous Goroutine to monitor ordered transactions, which are then proposed to the consensus algorithm by invoking the *propose* function from the BDLS core library.

V. BDLS PROTOCOL

See Protocol 1.

VI. EXPERIMENTAL EVALUATION

Integrating BDLS into the Fabric Orderer architecture aligns closely with the Raft Orderer implementation. It represents the pioneering instance of successful BFT protocol integration within the standardized Fabric Orderer architecture,

Protocol 1 BDLS (Normal Protocol Operation)

Inputs. For all $P_j \in [n]$, participant P_j propose a block B'.

Goal. All BDLS nodes (P_j, P_i) must agree on the submitted block B' from honest nodes at round r on height h.
The protocol: Request: The client submits the block B' to all BDLS nodes.

1) **Phase 1: Leader Propose** Each participant P_j (Includes leader P_i) sends a signed message $(\langle h, r\rangle_j, \langle h, r, B'_j\rangle_j)$ to the leader P_i, where $B'_j \in \text{BLOCK}_j$ represents the highest eligible candidate block for P_j. The message $(h, r)_j$ is designated as a round-change message. Following the transmission of the round-change message, P_j ceases to accept any further messages, except for a "decide" message on a round $r' < r$.
2) **Phase 2: Lock** P_i the leader receives messages from all consensus teams including himself. proof is a list of at least 2t+1 (nodes) same signed messages.
   a) P_i the leader receives from at least 2t + 1 participants with the same candidate block in signed messages. **Then:** All participants p_j and the leader p_i must receive this message from the leader p_i.
   $$\langle \text{lock}, h, r, B', \text{proof}\rangle_i$$
   A constructed digital signature on the message $\langle h, r, B'\rangle$ represents the "proof."
   b) If P_i the leader did not receive the block B ' from at least 2t + 1 participants. All candidate blocks sent to the leader p_i added in the local variable $BLOCK_i$.
   The leader P_i broadcasts best-learned block B''.
   $$\langle \text{select}, h, r, B'', \text{proof}\rangle_i$$
   $B'' = \max\{B : B \in \text{BLOCK}_i\}$.
   The "proof" contains an assembled digital signature on the message $\langle h, r\rangle$.
   A constructed digital signature on the message $\langle h, r\rangle$ represents the "proof".
3) **Phase 3: Commit** If All participants P_j (including the leader P_i) receives: $\langle \text{lock}, h, r, B', \text{proof}\rangle_i$ Then all participants P_j (including the leader P_i) send to the leader P_i.
   $$\langle \text{commit}, h, r, B'\rangle_j$$
   When participants receive: $\langle \text{select}, h, r, B'', \text{proof}\rangle_i$ Then participants adds $B'' \rightarrow \text{BLOCK}_j$
4) **Phase 4: Decide** The leader P_i receives $2t + 1$ commit messages then:
   a) P_i decides on the value B'
   b) leader P_i broadcast the decided message to all participants and himself.
   $$\langle \text{decide}, h, r, B', \text{proof}\rangle_i$$

constituting a novel consensus protocol amalgamation. The recent release of Fabric-BDLS allows system administrators to operate the network by choosing between BDLS Orderer and Raft-type Orderer nodes. BDLS offers the flexibility to coexist with Raft within a unified build, making it an adaptable choice. BDLS-Fabric is a valuable guide for integrating various consensus protocols, facilitated by its transparent Orderer implementation. The parallelism between BDLS and Raft Orderer implementations encompasses protocol initiation, message reception, order orchestration, batching, proposal submission to the core protocol, and block receipt, all within the same channel. Notably, the implementation within the Fabric core code remains transparent, devoid of concealed logic or migration of implementation details to the protocol code, enhancing the clarity and reliability of the integrated system.

A. BENCHMARK

The principle of TPS (Transactions Per Second) calculation for Hyperledger Caliper [27] is to measure the throughput, which is the total number of transactions divided by the total time in seconds [28].

The TPS calculation process begins by setting the start time immediately before calling the function responsible for creating the Envelope object. The end time is recorded after the last message is received and written into the block file. This ensures an accurate measurement of the time taken to process all transactions.

The benchmark measures TPS by determining the number of transactions processed per second.

$$\text{TPS} = \frac{\text{float64}(\textit{TotalTxNumber} \times \text{math.Pow}(10, 9))}{\text{float64}(\text{Tx submission time - block write time})}$$

where "total" represents the total time in nanoseconds taken to process 100,000 transactions. This approach ensures precise and reliable measurement of the TPS, which is critical for evaluating the performance of different consensus protocols in Hyperledger Fabric.

For example, during our experiments, we recorded the start time once creating the Tx to be sent and the end time after the last Tx message was written to the block. By applying the above formula, we obtained the TPS values, which provided insights into the efficiency of the consensus protocols under test.

By adopting this method, we can compare the performance of various consensus mechanisms, such as Raft, SmartBFT, and BDLS, under different network conditions.

B. EXPERIMENTAL SETUP

The operating system used for the experimental evaluation is Ubuntu 24.04 LTS, which is run on a system with 16GB of memory.

The expected delay is a common property for a distributed system protocol, known as latency. We set the latency for

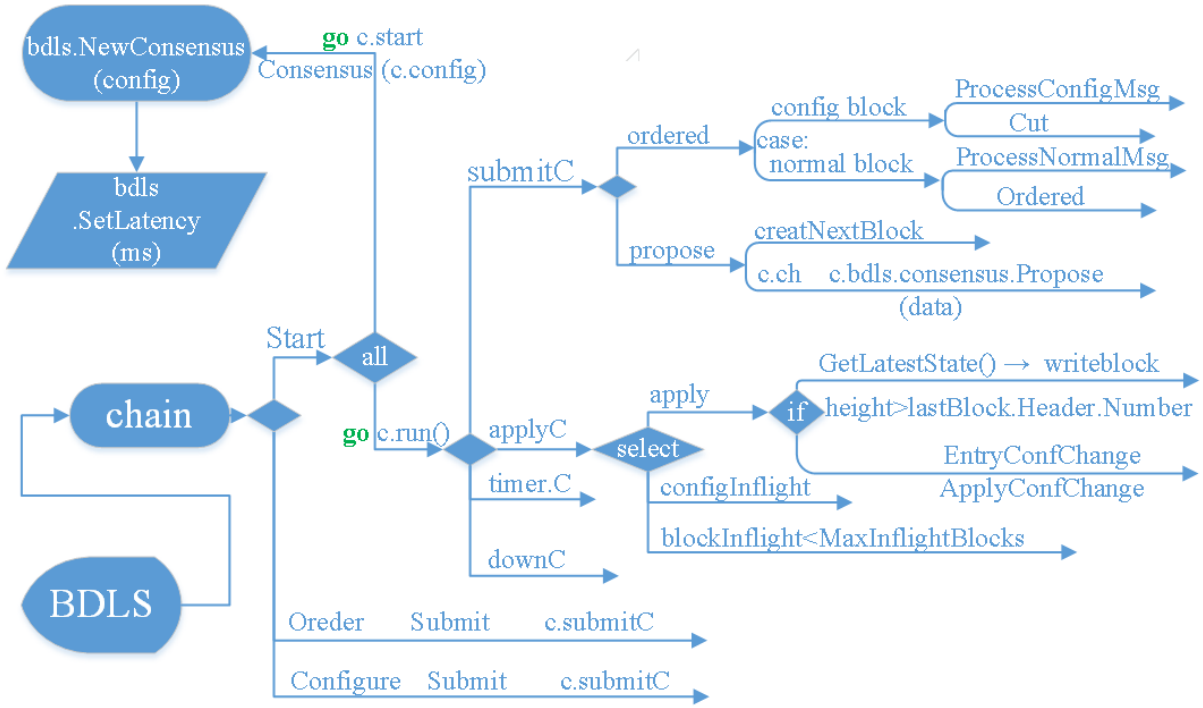


FIGURE 5. Block flow in BDLS-Fabric.

BDLS based on the average expected. The latency to the followers currently stands at an average of 100ms, with a minimum of 20 ms recorded in NYC and a maximum of 200 ms in Tokyo.

The experimental setup involves comparing the performance of the BDLS, Raft, and SmartBFT protocols in terms of Transactions Per Second (TPS) across different clusters and varying transactions per block. The results are represented in graphs as Figure 6 a, b, and c.

The Fabric Orderer configuration used for the benchmark in this paper is the same data matrix used by the latest Hyperledger published result [29]. The Fabric release operated in the tests is the latest main branch, including Orderer 3.0.

C. BDLS PERFORMANCE

The BDLS protocol, designed as a Byzantine Fault Tolerant (BFT) solution, showed remarkable performance in the experiments that were conducted. In various configurations, including clusters of 4, 5, and 6 nodes, BDLS achieves high TPS values across different transaction loads. For instance, with 1500 transactions per block, BDLS achieves up to 2860 TPS in a 4-node cluster, 2870 TPS in a 5-node cluster, and 2790 TPS in a 6-node cluster. These results indicate that BDLS can efficiently handle high transaction throughput while maintaining the robustness required for BFT systems.

When comparing Raft's performance to BDLS across different cluster setups, it was observed that BDLD TPS in Figure 6 (a) achieves around 89.76 % to 94.33 % of Raft's TPS in Figure 6 (b).

The overall architectural view of the Hyperledger Fabric network is depicted in Figure 7. The process begins with the client submitting a transaction to the peer node to which the client has access. The peer node then validates the transaction, which is subsequently proposed to the ordering service by the Software Development Kit (SDK). The ordering service is composed of a minimum of four BDLS nodes.

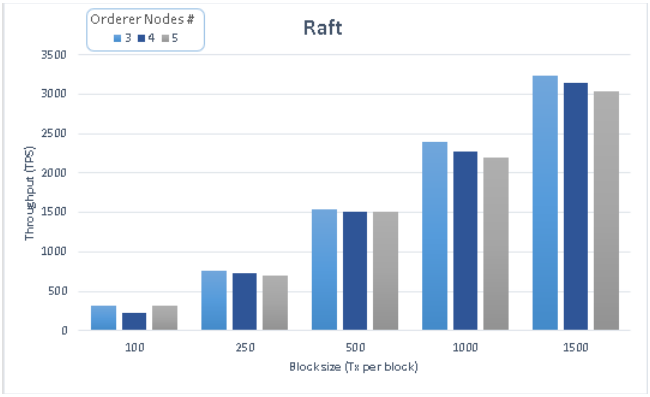
D. SMARTBFT/FABRIC 3.0 BFT PERFORMANCE

Our experimental results regarding SmartBFT almost match the published test results in [6] by the protocol developers "in a LAN, BFT-OS achieves 20% the performance of a Raft protocol (2,500 vs. 13,000 tx/sec, respectively in a WAN, SmartBft achieves 40% the performance of a Raft protocol (1,200 vs. 3,000 tx/sec, respectively." [6], [14].

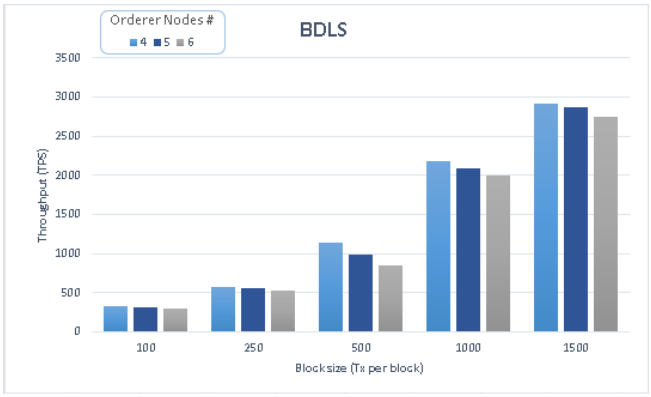
When compared to Raft's performance, SmartBFT in Figure 6 (c) achieves 33.33 % to 39.71 % of Raft's TPS in Figure 6 (b) across different clusters setup.

E. EXPERIMENTAL SUMMERY

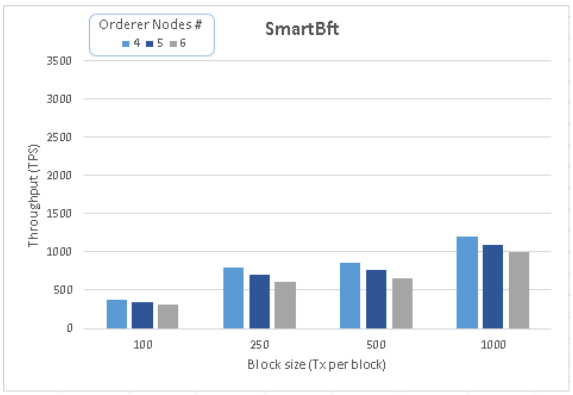
The evaluation demonstrates that BDLS is a highly efficient BFT solution for Hyperledger Fabric's ordering service, offering robustness and high throughput. Its performance is closely comparable to Raft, which is traditionally not designed for Byzantine Fault Tolerance but is highly efficient. SmartBFT, while providing BFT properties, shows lower TPS values compared to BDLS, highlighting BDLS's superior performance in high-throughput environments, as illustrated in Figure 8.



(a) Raft Experimental



(b) BDLS Experimental



(c) SmartBft Experimental(HLF Orderer 3.0)

FIGURE 6. Throughput (TPS) for Hyperledger Fabric network running: BDLS vs Raft vs Fabric Orderer v3.0 (smartBft).

VII. DISCUSSION

The novel architecture for interfacing with BDLS core functions is distinct, providing enhanced memory efficiency and performance optimization. It specifically utilizes *c.consensus.Update(time.Now())* and direct access to *c.consensus.CurrentState()* within an infinite loop in Hyperledger Fabric, replacing the conventional Go routine implementation. Furthermore, the periodic execution of these functions via a preset ticker timer acts as the heartbeat mechanism for BDLS. This design not only exemplifies an optimal framework for BDLS-Fabric integration, particularly in terms of performance efficiency and implementation efficacy, but it also aligns the interaction with BDLS functions in the *startConsensus* function with Fabric's standard functions, facilitating easier customization.

The experimental results clearly demonstrate that BDLS offers a substantial performance advantage over SmartBFT, with transaction per second (TPS) rates nearly on par with Raft. Specifically, BDLS's TPS values are only slightly lower than Raft's, making it a highly efficient Byzantine Fault Tolerant (BFT) solution. This efficiency is particularly significant for Hyperledger Fabric's ordering service, where both high throughput and robustness are essential.

The superior performance of BDLS over SmartBFT can be attributed to its optimized consensus mechanism, which effectively handles transaction validation and block generation while ensuring Byzantine Fault Tolerance. BDLS manages to achieve a balance between performance and fault tolerance, optimizing processes to maintain high TPS rates. In contrast, Raft, although not a BFT solution, excels in environments where Byzantine Fault Tolerance requirements are less stringent, thereby supporting higher TPS under similar conditions. Despite these promising results, there is still considerable room for enhancement. One potential improvement is transitioning BDLS nodes' communication from TCP connections to gRPC services. gRPC, with its advantages in performance and scalability, could further enhance the efficiency and reliability of BDLS's communication protocol.

In summary, our experimental results demonstrate the effectiveness of integrating the BDLS protocol into Hyperledger Fabric. The detailed steps and procedures for replicating these experimental tests are provided in Appendix A.

Furthermore, the integration of BDLS with Hyperledger Fabric shows significant potential for future growth. This integration is hosted as an open-source project in the Hyperledger Fabric lab, inviting contributions from the

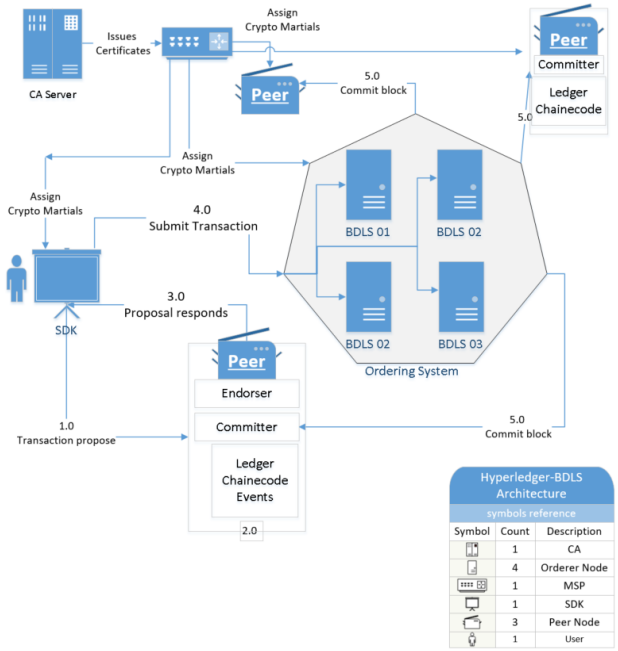


FIGURE 7. BDLS flow in Fabric.

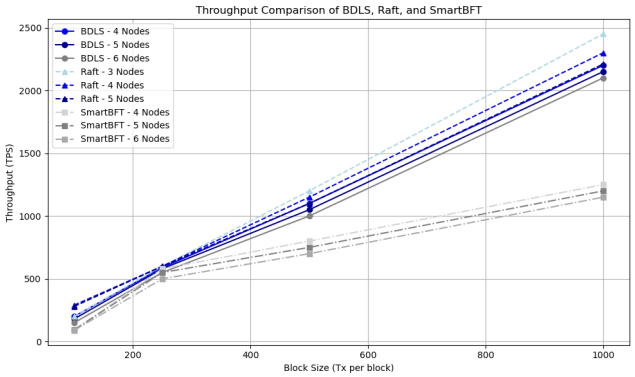


FIGURE 8. Throughput Comparison of BDLS, Raft, and SmartBFT.

community to enhance further and develop this promising protocol. Information on contributing is detailed in Appendix B.

VIII. CONCLUSION

In conclusion, our research underscores the limitations of Hyperledger Fabric's newly integrated SmartBFT protocol, particularly its high message complexity and scalability issues. The BDLS protocol, in contrast, demonstrates superior scalability and efficiency, maintaining linear message complexity regardless of network size. Our comparative analysis reveals that BDLS-based Fabric achieves comparable throughput to Raft-based Fabric while offering enhanced Byzantine fault tolerance. This makes BDLS a promising alternative for large-scale decentralized applications across various industries. Future work should further optimize BDLS integration and explore its performance in diverse real-world scenarios to validate its practical benefits. Our findings advocate for broader adoption of BDLS in Hyperledger Fabric to achieve more robust and scalable blockchain solutions.

BDLS advances the Hyperledger Fabric ecosystem by achieving linear message complexity, $O(n)$, unlike the quadratic, $O(n^2)$, in traditional BFT protocols, reducing communication overhead and enhancing scalability in a permissioned blockchain.

The new architecture for interacting with BDLS core functions enhances memory utilization and performance. Using *Update* function and direct access to *CurrentState* within a periodic infinite loop serves as the BDLS heartbeat, suggesting an optimal design for BDLS-Fabric integration in terms of efficiency and implementation.

APPENDIX A
EXPERIMENTAL TEST GUIDE

Refer to the guide in the BDLS repository for detailed instructions on running the experimental tests::

- For the BDLS Fabric Experimental Test Guide, visit: https://github.com/BDLS-bft/experiment-guide

APPENDIX B
CONTRIBUTING TO BDLS FABRIC

The BDLS Fabric integration proposal has been accepted and is now available in the Hyperledger Fabric lab repository. Refer to the following guidelines for contributing:

- For contribution guidelines, visit the CONTRIBUTING.md file at: https://github.com/hyperledger-labs/bdls

REFERENCES

[1] D. Ongaro and J. K. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annu. Tech. Conf.*, 2014, pp. 305–319.
[2] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. de Caro, D. Enyeart, C. Ferris, G. Laventman, and Y. Manevich, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, 2018, pp. 1–15.
[3] (2011). *Apache Kafka*. [Online]. Available: https://kafka.apache.org/
[4] (2023). *Hyperledger Fabric Consensus Protocols Release-2.5 Orderer/-Consensus Package*. [Online]. Available: https://github.com/hyperledger/fabric/tree/release-2.5/orderer/consensus
[5] C. Cachin, "Architecture of the hyperledger blockchain fabric," in *Proc. Workshop Distrib. Cryptocurrencies Consensus Ledgers*, Chicago, IL, USA, vol. 310, 2016, pp. 1–4.
[6] A. Barger, Y. Manevich, H. Meir, and Y. Tock, "A Byzantine fault-tolerant consensus library for hyperledger fabric," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, May 2021, pp. 1–9.
[7] Hyperledger Fabric. (2014). *Hyperledger Fabric V3.0*. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/whatsnew.html
[8] *Pluggable Consensus*. Accessed: Sep. 20, 2024. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/whatis.html#pluggable-consensus
[9] (2015). *Openblockchain*. [Online]. Available: https://github.com/openblockchain
[10] Binh Nguyen. (2017). *Hyperledger Fabric—A Brief History*. [Online]. Available: https://www.linkedin.com/pulse/hyperledger-fabric-brief-history-binh-nguyen/?trackingId=npAL2EodQECJPdouPaw59A%3D%3D
[11] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. OsDI*, vol. 99, 1999, pp. 173–186.

[12] Q. Nasir, I. A. Qasse, M. Abu Talib, and A. B. Nassif, "Performance analysis of hyperledger fabric platforms," *Secur. Commun. Netw.*, vol. 2018, pp. 1–14, Sep. 2018.
[13] H. Sukhwani, N. Wang, K. S. Trivedi, and A. Rindos, "Performance modeling of hyperledger fabric (permissioned blockchain network)," in *Proc. IEEE 17th Int. Symp. Netw. Comput. Appl. (NCA)*, Nov. 2018, pp. 1–8.
[14] J. Sousa, A. Bessani, and M. Vukolic, "A Byzantine fault-tolerant ordering service for the hyperledger fabric blockchain platform," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2018, pp. 51–58.
[15] (2024). *Hyperledger Fabric V3.0*. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/whatsnew.html
[16] M. Yin, D. Malkhi, M. K. Reiter, G. Golan Gueta, and I. Abraham, "HotStuff: BFT consensus in the lens of blockchain," 2018, *arXiv:1803.05069*.
[17] M. Baudet, A. Ching, A. Chursin, G. Danezis, F. Garillot, Z. Li, D. Malkhi, O. Naor, D. Perelman, and A. Sonnino, "State machine replication in the libra blockchain," Libra Assoc., Geneva, Switzerland, Tech. Rep. 7, 2019.
[18] Y. Wang, "Byzantine fault tolerance for distributed ledgers revisited," *Distrib. Ledger Technol., Res. Pract.*, vol. 1, no. 1, pp. 1–26, Sep. 2022.
[19] E. Buchman, J. Kwon, and Z. Milosevic, "The latest gossip on BFT consensus," 2018, *arXiv:1807.04938*.
[20] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," in *Proc. Concurrency: Works Leslie Lamport*, 2019, pp. 203–226.
[21] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the presence of partial synchrony," *J. ACM*, vol. 35, no. 2, pp. 288–323, Apr. 1988.
[22] Y. Wang, "Byzantine fault tolerance in partially synchronous networks," Cryptol. ePrint Arch., Tech. Rep., 2019.
[23] *BDLS Fabric Project Repository*. Accessed: May 1, 2023. [Online]. Available: https://github.com/BDLS-bft/fabric
[24] *BDLS Fabric Samples/Test Network Repository*. Accessed: Sep. 25, 2023. [Online]. Available: https://github.com/BDLS-bft/fabric-samples
[25] *BDLS Fabric SDK Repository*. Accessed: Jul. 20, 2023. [Online]. Available: https://github.com/BDLS-bft/fabric-sdk-java
[26] (2024). *Orderer Server Main.Go Source Code File BDLS Consensus Protocol Github Organization*. [Online]. Available: https://github.com/BDLS-bft/fabric/blob/main-bdls/orderer/common/server/main.go
[27] M. Shuaib, N. H. Hassan, S. Usman, S. Alam, N. A. A. Bakar, and N. Maarop, "Performance evaluation of DLT systems based on hyper ledger fabric," in *Proc. 4th Int. Conf. Smart Sensors Appl. (ICSSA)*, Jul. 2022, pp. 70–75.
[28] W. Choi and J. W. Hong, "Performance evaluation of Ethereum private and testnet networks using hyperledger caliper," in *Proc. 22nd Asia–Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2021, pp. 325–329.
[29] Hyperledger. (2023). *Benchmarking Hyperledger Fabric 2.5 Performance*. [Online]. Available: https://www.hyperledger.org/blog/2023/02/16/benchmarking-hyperledger-fabric-2-5-performance

AHMED AL SALIH (Member, IEEE) received the M.Sc. degree in information technology from Southern New Hampshire University, in 2014. He is currently pursuing the Ph.D. degree with the Department of Software and Information Systems, The University of North Carolina at Charlotte, under the guidance of Dr. Yongge Wang.

Since 2014, he has been a Senior Software Engineer with Ahold Delhaize, with over nine years of experience as a Senior Software Engineer and Tech Lead. His research focuses on integrating Byzantine fault tolerance protocols within Hyperledger Fabric. He brings a wealth of practical knowledge to his academic pursuits. His dedication to delivering innovative solutions and tackling challenges serves as a driving force behind his research endeavors. He has focused on engineering and shipping new systems, delivering solutions, and modernizing enterprise applications. He has held various positions in the IT field in both industry and academia more than the past 20 years, focusing on researching, mentoring individuals, and delivering solutions. His research interests include blockchain consensus mechanisms, the IoT, and enhancing the security and performance of distributed ledger technologies.

YONGGE WANG (Member, IEEE) received the M.Sc. degree from the S. S. Chern Institute of Mathematics, Nankai University, China, in 1991, and the Ph.D. degree in computer science from Heidelberg University, Germany, in 1996.

He is currently a Professor with the Department of Software and Information Systems (SIS), University of North Carolina at Charlotte, specializing in algorithmic complexity and cryptography. He is the Inventor of IEEE P1363 Cryptographic Standards SRP5 and WANG-KE and has contributed to the mathematical theory of algorithmic randomness. He has co-authored an article demonstrating that a recursively enumerable real number is an algorithmically random sequence if and only if it is a Chaitin's constant for some encoding of programs. He also showed the separation of Schnorr randomness from recursive randomness. He also invented a distance based statistical testing technique to improve NIST SP800-22 testing in randomness tests. In cryptographic research, he is known for the invention of the quantum resistant random linear code based encryption scheme RLCE.